



Apuntes de Redes de Datos

Parte 2 — Nivel Senior

SDN, Datacenter, Cloud, Kubernetes, MPLS, Multicast

Aldo Zepeda Moctezuma

18 de mayo de 2026

“La infraestructura que no entiendes es la infraestructura que te va a fallar a las 3am.”

— Sabiduría de SRE

Índice general

1. 🚀	Introducción a la Parte 2 — Nivel Senior	7
1.1.	📖 ¿Qué cubre esta parte?	7
1.2.	🏢 El Stack del Datacenter Moderno	8
1.3.	🗺 Mapa de Tecnologías	9
1.4.	⚖ Parte 1 vs Parte 2 — ¿Qué cambia?	9
1.5.	🎓 ¿Por qué importa todo esto?	10
2. 📶	Wireless Empresarial — WPA3 y 802.1X	13
2.1.	🔒 WPA3 — Wi-Fi Protected Access 3	13
2.1.1.	🔑 SAE — Simultaneous Authentication of Equals	14
2.1.2.	🌐 OWE — Opportunistic Wireless Encryption	15
2.2.	🛡 802.1X — Port-Based Network Access Control	16
2.2.1.	🔗 EAP — Extensible Authentication Protocol	16
2.2.2.	🔧 RADIUS — Remote Authentication Dial-In User Service	17
2.2.3.	📄 Flujo Completo 802.1X — Diagrama de Secuencia	18
3. 🛠	Switching Avanzado	21
3.1.	🔄 MSTP — Multiple Spanning Tree Protocol (802.1s)	21
3.1.1.	Conceptos clave	21
3.1.2.	Configuración Cisco IOS	22
3.1.3.	Diagrama: MSTP balanceando tráfico	22
3.2.	🔒 MACsec — MAC Security (802.1AE)	22
3.2.1.	Cómo funciona	22
3.2.2.	Formato del frame MACsec	23
3.2.3.	TLS vs MACsec	23
3.3.	🏠 QinQ — Double Tagging (802.1ad)	23
3.3.1.	Formato del frame QinQ	24
3.4.	🏠 Private VLANs	24
3.4.1.	Tipos de puertos	24
3.5.	📶 IGMP Snooping	25
3.5.1.	Flujo de aprendizaje	25
3.6.	☁ Storm Control	26
3.7.	🔑 802.1X en Switches — Wired NAC	26
3.7.1.	MAB — MAC Authentication Bypass	26
3.7.2.	VLANs de contingencia	27

4.		SDN y OpenFlow	29
4.1.		El Problema del Networking Tradicional	29
4.2.		Arquitectura SDN	29
4.2.1.		Las tres capas	30
4.2.2.		Northbound vs Southbound API	30
4.3.		OpenFlow Protocol	30
4.3.1.		Flow Entry: Match + Actions	30
4.3.2.		Formato de flow entry OpenFlow 1.3	30
4.3.3.		Pipeline de tablas	31
4.4.		Controladores SDN	31
4.4.1.		OVS — Open vSwitch	31
4.5.		Intent-Based Networking (IBN)	32
4.6.		SDN en Producción: Realidades	32
5.		Datacenter: VxLAN y EVPN	33
5.1.		Limitaciones de VLANs en Datacenter	33
5.2.		VxLAN — Virtual Extensible LAN	33
5.2.1.		Formato del frame VxLAN	34
5.3.		Topología Spine-Leaf	34
5.3.1.		Reglas del diseño Spine-Leaf	34
5.3.2.		Diagrama Spine-Leaf con VxLAN	35
5.4.		EVPN — Ethernet VPN	35
5.4.1.		Route Types de EVPN	35
5.4.2.		Configuración básica (Cisco NX-OS)	36
5.5.		Routing Asimétrico vs Simétrico en VxLAN	36
6.		MPLS y QoS	39
6.1.		MPLS — Multiprotocol Label Switching	39
6.1.1.		Formato de la etiqueta MPLS	39
6.2.		Operaciones de etiquetas	40
6.3.		MPLS Traffic Engineering (TE)	40
6.4.		QoS — Quality of Service	41
6.4.1.		Los tres problemas que QoS resuelve	41
6.4.2.		Modelo DiffServ y DSCP	41
6.4.3.		Mecanismos de QoS	42
7.		Networking en Kubernetes	45
7.1.		El Modelo de Red de Kubernetes	45
7.2.		Tipos de red en un cluster	46
7.3.		Services y kube-proxy	46
7.3.1.		Tipos de Service	46
7.4.		Network Policies	47
7.5.		Ingress y Gateway API	48
7.6.		CNI Plugins: Cilium vs Calico	49
8.		Cloud Networking	51
8.1.		VPC — Virtual Private Cloud	51
8.1.1.		Arquitectura típica multi-tier	52
8.2.		Conectividad entre VPCs y On-Premise	52

8.2.1.	 VPC Peering	52
8.2.2.	Transit Gateway / Cloud Router	52
8.2.3.	VPN y Direct Connect / Dedicated Interconnect	52
8.3.	 Load Balancing en la nube	53
8.4.	 Seguridad de Red en la Nube	53
8.4.1.	Security Groups vs NACLs	53
8.4.2.	AWS PrivateLink / GCP Private Service Connect	54
8.5.	 CDN y Edge Networking	54
9.	 Multicast	55
9.1.	 Unicast vs Broadcast vs Multicast	55
9.2.	 Direcciones Multicast	55
	9.2.1. MAC Multicast	56
9.3.	 IGMP — Gestión de Grupos	56
9.4.	 PIM — Routing Multicast entre Routers	57
	9.4.1. PIM Dense Mode (PIM-DM)	57
	9.4.2. PIM Sparse Mode (PIM-SM)	57
	9.4.3. PIM Source-Specific Multicast (PIM-SSM)	57
9.5.	 Aplicaciones Reales	58
9.6.	 Resumen del Ecosistema Parte 2	58

Capítulo 1

Introducción a la Parte 2 — Nivel Senior

1.1. ¿Qué cubre esta parte?

Concepto

La **Parte 1** de estos apuntes cubrió los **fundamentos de redes**: el modelo OSI, TCP/IP, protocolos clave (TCP, UDP, DNS, HTTP, TLS, SSH) y cómo fluyen los datos de una máquina a otra.

La **Parte 2** sube un nivel. Aquí vemos cómo funcionan las redes **por dentro**, en producción real, a escala empresarial y de nube:

- **Datacenters modernos**: VxLAN, EVPN, Spine-Leaf, SDN
- **Redes de operador (carrier)**: MPLS, QoS, ingeniería de tráfico
- **Kubernetes**: CNI, Service Mesh, Ingress, eBPF
- **Cloud networking**: AWS VPC, Transit Gateway, Direct Connect
- **Seguridad de switch empresarial**: 802.1X, MACsec, VLAN hardening
- **Wi-Fi empresarial**: WPA3-Enterprise, 802.11ax (Wi-Fi 6)
- **Multicast**: PIM, IGMP, streaming a gran escala

 **Analogía**: La Parte 1 es aprender a **manejar un auto**: sabes presionar el acelerador, girar el volante, respetar semáforos. La Parte 2 es aprender a **construir el motor**: sabes cómo funciona el sistema de combustión, la transmisión, el turbocompresor — y por qué falla a las 3am cuando hay 50,000 usuarios conectados.

1.2. 🏢 El Stack del Datacenter Moderno

Antes de entrar a cada tecnología, necesitas ver el panorama completo. Este es el flujo real de un paquete desde Internet hasta el contenedor que lo procesa en un datacenter moderno (AWS, Google Cloud, un banco grande):

```
Internet
|
v
Edge Routers  (BGP - hablan con todo Internet)
|
v
Firewall / DDoS Mitigation  (filtra ataques, limita flujos)
|
v
Load Balancers  (L4/L7 - distribuyen tráfico entre servidores)
|
v
Spine Switches  (ECMP - múltiples rutas iguales, sin STP)
|
v
Leaf Switches  (VxLAN VTEP - virtualizan la red sobre IP)
|
v
Servers / Hypervisors  (Linux + Open vSwitch o eBPF)
|
v
Pods / Containers  (Kubernetes - la app vive aquí)
```



Concepto

Cada flecha del diagrama es un capítulo de esta parte:

- **BGP** se estudia en Cloud Networking (cap. 07).
- **Firewall / DDoS** es parte de Seguridad empresarial (cap. 02).
- **Load Balancers + VxLAN + Spine-Leaf** son el cap. 04 (Datacenter).
- **MPLS y QoS** mueven paquetes entre datacenters (cap. 05).
- **Pods y Kubernetes** son el cap. 06.



Analogía: El datacenter es como un aeropuerto enorme. Internet es el mundo exterior. Los Edge Routers son la *aduana*. Los Spine Switches son las *pistas de aterrizaje* (no hay semáforos — todo va en paralelo). Los Leaf Switches son las *puertas de abordaje*. Los Pods son *los asientos de cada vuelo*: el destino final del pasajero (paquete).

1.3. Mapa de Tecnologías

La siguiente tabla muestra qué cubre cada capítulo, qué tecnologías involucra y en qué entorno vive en producción real:

Cap.	Tecnología	Dónde vive
01	Wi-Fi Empresarial: 802.11ax, WPA3- Enterprise, RADIUS, roaming 802.11r	Oficinas corporativas, campus universitarios, hoteles, aeropuertos
02	Switching Avanzado: VLANs, STP/RSTP, 802.1X, MACsec, port security	Cualquier red con switches administrables (empresa, datacenter, banco)
03	SDN y OpenFlow: controladores, APIs northbound/south- bound, OpenDaylight	Datacenters de hiperescaladores (Google, Facebook), redes universitarias
04	Datacenter: VxLAN, EVPN, Spine-Leaf, ECMP, BGP-EVPN	AWS, Azure, GCP, Equinix, datacenters privados grandes
05	MPLS y QoS: LSP, LDP, RSVP-TE, ingeniería de tráfico, DiffServ	Redes de operador (Telmex, AT&T, Lumen), troncales entre datacenters
06	Kubernetes Networ- king: CNI, Calico, Cilium/eBPF, Service Mesh, Ingress	Toda empresa que corre micro-servicios en producción
07	Cloud Networking: VPC, Transit Gateway, Direct Connect, BGP con AWS	AWS, Azure, GCP — el 90 % de las startups y empresas medianas
08	Multicast: PIM-SM, PIM-SSM, IGMP, MDT, streaming y finanzas	Broadcasters (televisión IP), bolsas de valores, CDNs

1.4. Parte 1 vs Parte 2 — ¿Qué cambia?

Esta tabla compara el nivel de cada tema entre las dos partes. No es que Parte 1 sea “incorrecto” — es que Parte 2 es lo que se usa en producción a escala:

Dimensión	Parte 1 (Fundamentos)	Parte 2 (Producción)
Protocolos de transporte	TCP / UDP básico, puertos, handshake	MPLS / VxLAN, túneles, encapsulación multi-capa
Seguridad	TLS (HTTPS), SSH, firewalls básicos	MACsec (capa 2), 802.1X, WPA3-Enterprise, Zero Trust
Wi-Fi	802.11 básico, WPA2 personal, canales	Wi-Fi 6 (802.11ax), roaming empresarial, RADIUS/EAP
Enrutamiento	RIP, OSPF básico, tablas de rutas	BGP completo, MPLS-TE, ECMP en Spine-Leaf
Switches	VLANs básicas, STP para evitar loops	EVPN, VxLAN VTEP, MLAG, port security avanzado
Virtualización de red	No aplica (redes físicas)	SDN, OpenFlow, OVS, eBPF, Cilium
Contenedores	No aplica	CNI plugins, Kubernetes networking, Service Mesh
Cloud	No aplica	VPC, Transit Gateway, Direct Connect, Cloud Router
Escala	Red de laboratorio o SOHO (10–100 hosts)	Miles de servidores, millones de flujos simultáneos

1.5. 🎓 ¿Por qué importa todo esto?

🚀 Tecnología Moderna

El 90 % de la infraestructura empresarial moderna corre sobre:

- **Kubernetes** para orquestar contenedores
- **Cloud networking** (AWS VPC, Azure VNet) para conectar todo
- **MPLS** en las redes de operador que unen datacenters
- **VxLAN + EVPN** dentro del datacenter para virtualizar la red
- **802.1X + MACsec** para seguridad de acceso en campus empresariales

Las entrevistas de SRE / Network Engineer en empresas como Google, Amazon, Banco de México, BBVA, Telmex y cualquier empresa con más de 500 empleados preguntan exactamente estos temas.

🧠 Si sabes explicar por qué VxLAN usa UDP port 4789, cómo Kubernetes asigna IPs a los pods, y qué hace un LSR en MPLS — ya estás en el top 10 % de candidatos.

💡 **Analogía final:** Un médico general sabe que el corazón bombea sangre (Parte 1). Un cirujano cardiovascular sabe hacer un bypass, reparar una válvula y gestionar la UCI post-operatoria (Parte 2). Ambos son necesarios, pero el hospital solo le confía el quirófano al segundo.

Capítulo 2

Wireless Empresarial — WPA3 y 802.1X

Concepto

El Wi-Fi empresarial moderno no es el mismo router de casa con una contraseña compartida. En entornos corporativos, aeropuertos, universidades y hospitales, la seguridad inalámbrica se construye sobre tres pilares: **autenticación individual, cifrado por sesión y control de acceso por puerto**. Este capítulo cubre **WPA3** (el protocolo de seguridad Wi-Fi más moderno) y **802.1X** (el estándar de control de acceso a red por puerto, usado tanto en Wi-Fi como en switches cableados).

2.1. WPA3 — Wi-Fi Protected Access 3

Concepto

WPA2 (802.11i, 2004) usa **PSK** (Pre-Shared Key): una sola contraseña compartida que todos los usuarios conocen. Quien sabe la contraseña entra. Si esa contraseña se filtra, *toda la red queda comprometida*. Además, un atacante puede capturar el 4-way handshake y atacarlo offline con diccionario (Hashcat a millones de intentos por segundo).

WPA3 (2018) reemplaza PSK por **SAE** (Simultaneous Authentication of Equals), también conocido como “Dragonfly Handshake”. Cada cliente se autentica individualmente con un protocolo de prueba de conocimiento cero.

Analogía:

- **WPA2-PSK** = llave maestra que todos los empleados tienen. Si un empleado la pierde (o la fotografía), cualquiera entra.
- **WPA3-SAE** = tarjeta inteligente individual por empleado. Cada tarjeta es única; perder una no compromete las demás. Además, la tarjeta no “revela” el PIN al sistema — solo demuestra que lo conoce (prueba de conocimiento cero).

Característica	WPA2	WPA3
Intercambio de llaves	PSK (Pre-Shared Key) — contraseña compartida	SAE / Dragonfly — Diffie-Hellman con prueba de conocimiento cero
Forward Secrecy	No — capturar el handshake permite descifrar sesiones pasadas	Sí — cada sesión genera llaves efímeras únicas
Ataques offline	Vulnerable — atacante captura el 4-way handshake y ataca con diccionario	Resistente — SAE no permite ataques offline de diccionario
Redes abiertas	Sin cifrado (aeropuertos, cafeterías en texto claro)	OWE (Opportunistic Wireless Encryption) — cifrado sin contraseña
Modo empresarial	WPA2-Enterprise con 802.1X/EAP	WPA3-Enterprise con 802.1X/EAP + Suite B (criptografía de 192 bits)
Estándar IEEE	802.11i (2004)	IEEE 802.11-2020 (enmienda)

2.1.1. 🔑 SAE — Simultaneous Authentication of Equals

🧠 Concepto

SAE (RFC 7664 / IEEE 802.11-2020) es un protocolo de establecimiento de llaves basado en **Diffie-Hellman sobre curva elíptica** con una **prueba de conocimiento cero** (*zero-knowledge proof*).

¿Qué significa prueba de conocimiento cero aquí? En WPA2, durante el 4-way handshake, el cliente y el AP derivan una PMK a partir de la contraseña y la intercambian de forma que un atacante que capture el tráfico puede verificar contraseñas offline. En SAE, el protocolo demuestra que *ambas partes conocen la misma contraseña* sin que ninguna la transmita — ni en texto claro ni en forma derivada atacable offline.

Forward Secrecy (Secreto hacia adelante): Cada sesión Wi-Fi negocia un par de llaves efímeras (de un solo uso). Si un atacante graba todo el tráfico cifrado de hoy y mañana roba la contraseña del AP, *no puede descifrar el tráfico de hoy* porque las llaves efímeras ya no existen. WPA2 no tiene esta propiedad.

 Cuidado**KRACK Attack (Key Reinstallation Attack, 2017) — CVE-2017-13077**

Mathy Vanhoef descubrió que en el 4-way handshake de WPA2, un atacante Man-in-the-Middle podía forzar la **reinstalación de llaves ya usadas** manipulando los mensajes del handshake. Al reinstalar una llave con un nonce repetido, el cifrado CCMP se debilita drásticamente — en algunos casos hasta recuperación de texto claro.

Impacto: afectó prácticamente todo dispositivo Wi-Fi con WPA2 del planeta. **WPA3/SAE es inmune por diseño:** SAE no usa un 4-way handshake con reinstalación de llaves. El protocolo Dragonfly genera llaves únicas por sesión que no pueden ser reinstaladas, eliminando la clase de vulnerabilidad que KRACK explotó.

💡 Parches de WPA2 existen, pero WPA3 es la solución estructural.

2.1.2. 🌐 OWE — Opportunistic Wireless Encryption

 Concepto

OWE (RFC 8110) resuelve un problema clásico: las redes Wi-Fi abiertas (aeropuertos, cafeterías, hoteles) no tienen contraseña, por lo que el tráfico viaja en texto claro y cualquier persona con un adaptador en modo monitor puede **sniffear pasivamente** todo lo que transmite.

OWE mantiene la experiencia de usuario (no pide contraseña, te conectas directamente) pero cifra automáticamente el enlace inalámbrico mediante un intercambio **Diffie-Hellman** entre cliente y AP:

1. Cliente y AP intercambian claves públicas efímeras DH durante la asociación.
2. Derivan una **PMK** (Pairwise Master Key) compartida.
3. El 4-way handshake estándar procede para generar llaves de sesión.
4. Todo el tráfico queda cifrado con AES-CCM, incluso sin contraseña.

 **OWE** protege contra **sniffing pasivo** (alguien escucha tu tráfico Wi-Fi). No autentica al AP — un Evil Twin todavía puede responder. Para autenticación del AP, se necesita 802.1X o certificados adicionales.

2.2. 🛡️ 802.1X — Port-Based Network Access Control

🧠 Concepto

802.1X (IEEE 802.1X-2020) es el estándar de **control de acceso a red basado en puerto**. Define tres actores:

- **Supplicant** (Suplicante): el cliente que quiere acceso a la red (laptop, teléfono, impresora). Implementado en el sistema operativo (wpa_supplicant en Linux, native supplicant en Windows/macOS).
- **Authenticator** (Autenticador): el switch o Access Point. Tiene el puerto en estado **UNAUTHORIZED** (bloqueado) hasta que el servidor RADIUS aprueba al cliente. Solo deja pasar tráfico EAP (EAPoL) durante la autenticación.
- **Authentication Server** (Servidor de Autenticación): típicamente un servidor **RADIUS** (FreeRADIUS, Cisco ISE, Microsoft NPS). Verifica las credenciales y devuelve Access-Accept o Access-Reject.

💡 **Analogía:** el sistema de llave magnética de un hotel.

- El **Supplicant** es el huésped que quiere entrar a su habitación.
- El **Authenticator** es la cerradura electrónica de la puerta. No importa que el huésped sepa el número de habitación (contraseña compartida): sin la tarjeta válida, la puerta no abre.
- El **RADIUS Server** es el sistema central del hotel que verifica si esa tarjeta corresponde a una reserva activa y autoriza el acceso.

Si la tarjeta no es válida (credenciales incorrectas, certificado expirado, dispositivo no administrado), el puerto queda en UNAUTHORIZED y el cliente solo puede acceder a una VLAN de cuarentena — si es que eso se configura.

2.2.1. 🧪 EAP — Extensible Authentication Protocol

EAP (RFC 3748) es el protocolo de autenticación que viaja dentro de 802.1X. Es “extensible” porque define solo el *marco* del intercambio de autenticación; el mecanismo real es el **método EAP** elegido:

Método	Mecanismo	Caso de uso	Estado
EAP-TLS	Certificados X.509 mutuos (cliente y servidor)	Máxima seguridad; dispositivos administrados con PKI propia	Recomendado
PEAP	Contraseña dentro de túnel TLS (solo el servidor tiene certificado)	Integración con Active Directory; BYOD con credenciales de dominio	Aceptable
EAP-TTLS	Contraseña dentro de túnel TLS con PAP/CHAP interno	Linux, Android, BYOD heterogéneo; no requiere certificado en el cliente	Aceptable
EAP-MD5	Hash MD5 de la contraseña (sin cifrado del canal)	Legacy; sin protección contra ataques de diccionario offline	Obsoleto — No usar
EAP-FAST	PAC (Protected Access Credential) en lugar de certificado	Alternativa a PEAP en entornos Cisco sin PKI completa	Uso limitado

2.2.2. 📡 RADIUS — Remote Authentication Dial-In User Service

RADIUS (RFC 2865/2866) es el protocolo de AAA (Authentication, Authorization, Accounting) más usado en redes empresariales. Opera sobre UDP:

Puertos RADIUS:

```
UDP 1812 - Authentication (sustituye al histórico 1645)
UDP 1813 - Accounting     (sustituye al histórico 1646)
```

Mensajes principales:

```
Authenticator --> RADIUS: Access-Request
                        (contiene: User-Name, NAS-IP, EAP-Message,
                        Message-Authenticator [HMAC-MD5])
```

```
RADIUS --> Authenticator: Access-Accept
                        (+ atributos: Tunnel-Type=VLAN,
                        Tunnel-Medium-Type=802,
                        Tunnel-Private-Group-ID=<VLAN-ID>)
```

```
Access-Reject
(acceso denegado, puede incluir Reply-Message)
```

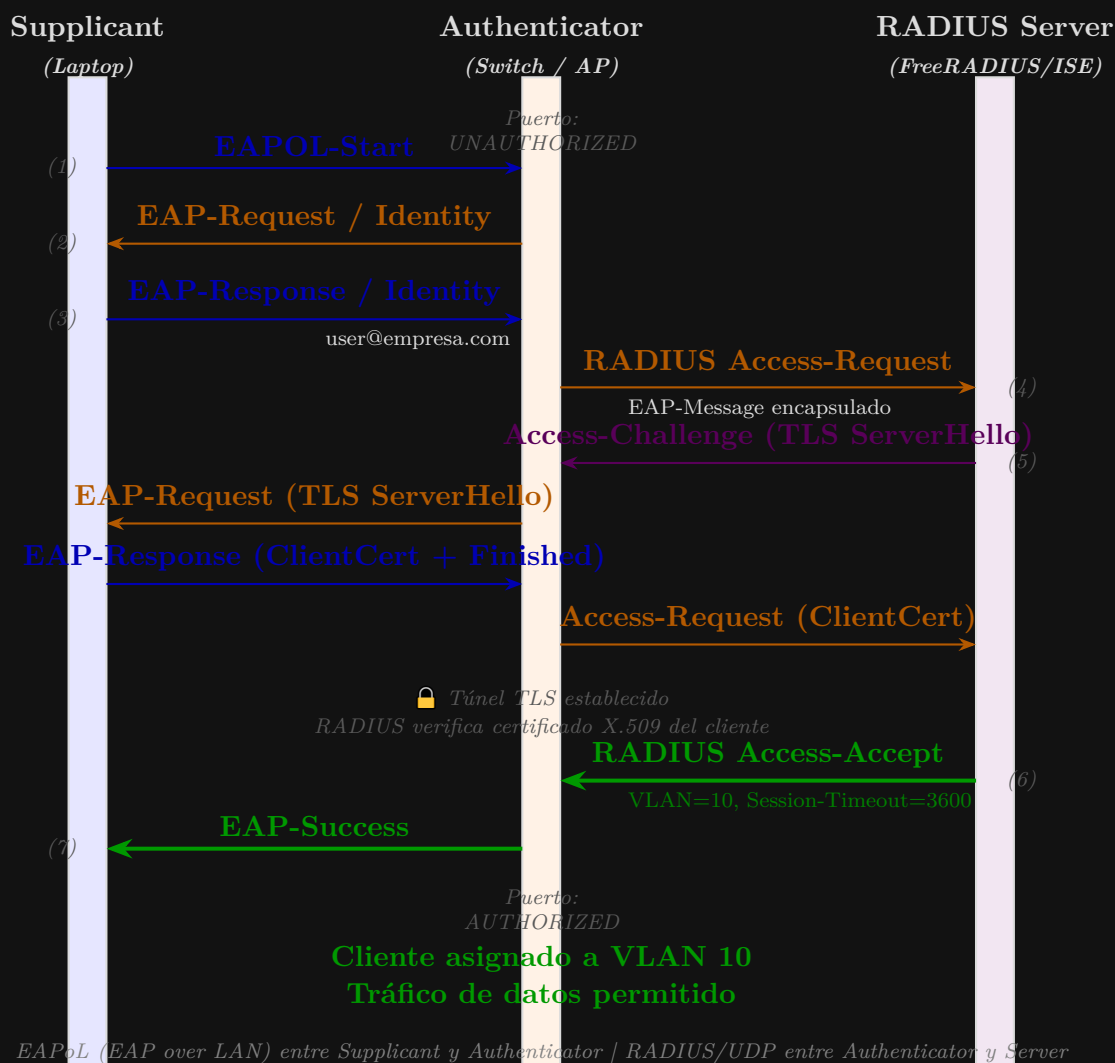
Access-Challenge
(solicita más información al cliente,
p.ej. segundo factor,
certificado)

Seguridad:

- El secreto compartido (RADIUS shared secret) cifra los atributos sensibles (contraseñas) usando MD5.
- Para producción: usar RADSEC (RADIUS over TLS, RFC 6614) en lugar de RADIUS sobre UDP plano.

2.2.3. 📡 Flujo Completo 802.1X — Diagrama de Secuencia

El siguiente diagrama muestra el flujo completo de autenticación 802.1X con EAP-TLS entre los tres actores:



 Tecnología Moderna**Google BeyondCorp — El dispositivo como identidad**

La implementación de 802.1X en Google va más allá del usuario. En el modelo **BeyondCorp** (el origen del término “Zero Trust”), la autenticación de red usa **EAP-TLS** donde el certificado pertenece al **DISPOSITIVO**, no al usuario:

- Cada laptop, teléfono y servidor administrado por Google recibe un **certificado X.509 único** emitido por la PKI interna de Google.
- El 802.1X valida ese certificado de dispositivo, no una contraseña de usuario.
- Si un empleado trae su laptop personal (*dispositivo no administrado*), el AP la rechaza en el 802.1X — no hay RADIUS Access-Accept — independientemente de que el empleado tenga credenciales válidas de usuario.
- **Credenciales de usuario robadas + dispositivo no administrado = cero acceso a la red interna.**

🧠 Este modelo invierte el supuesto de confianza tradicional: en vez de “confío en quien está dentro de la red”, el modelo es “*no confío en nadie hasta verificar dispositivo, usuario y contexto*”. La red es ahora solo un canal de transporte — el acceso real lo controla la capa de aplicación con certificados y políticas de dispositivo.

Capítulo 3

Switching Avanzado

Si en la Parte 1 aprendiste que un switch aprende MACs y reenvía tramas, aquí vas a aprender cómo los switches de nivel empresarial manejan escenarios que un switch hogareño ni soñaría: múltiples instancias de Spanning Tree, cifrado en el cable, VLANs anidadas, aislamiento de puertos, control de tormentas de tráfico, y autenticación de dispositivos en el puerto físico.

3.1. MSTP — Multiple Spanning Tree Protocol (802.1s)

Concepto

Problema con RSTP clásico: una sola instancia STP para *todas* las VLANs. Eso significa que si el árbol bloquea un enlace, lo bloquea para **todo el tráfico**, sin importar la VLAN. Es como cerrar un carril de una autopista y afectar a todos los vehículos, aunque la mitad fueran por destinos distintos. **MSTP (802.1s)** mapea grupos de VLANs a instancias STP independientes, cada una con su propio árbol y su propio Root Bridge. Resultado: diferentes VLANs pueden usar diferentes caminos físicos de forma simultánea.

3.1.1. Conceptos clave

- **MST Region:** conjunto de switches con la misma configuración MST (mismo nombre de región, número de revisión y mapeo VLAN→instancia). Los switches fuera de la región se ven como un único switch virtual (boundary).
- **IST (Internal Spanning Tree / Instancia 0):** instancia especial que interopera con switches RSTP/STP clásicos en la frontera de la región.
- **MSTI (MST Instance):** instancias 1–64, cada una con su propio Root Bridge y topología independiente.

3.1.2. Configuración Cisco IOS

📁 Configuración / Código

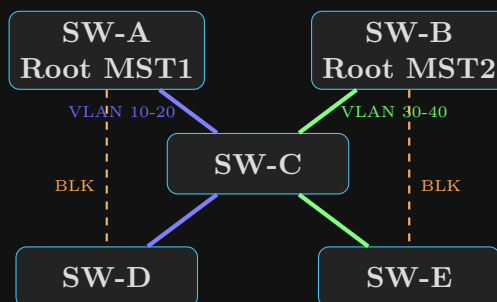
```
spanning-tree mode mst

spanning-tree mst configuration
  name CORP_REGION
  revision 1
  instance 1 vlan 10-20
  instance 2 vlan 30-40
  exit

! Root bridge para instancia 1
spanning-tree mst 1 priority 4096

! Root bridge para instancia 2
spanning-tree mst 2 priority 4096
```

3.1.3. Diagrama: MSTP balanceando tráfico



VLAN 10-20 usan el camino izquierdo; VLAN 30-40 el derecho. Cada instancia bloquea enlaces distintos: **cero desperdicio de ancho de banda**.

3.2. 🗝️ MACsec — MAC Security (802.1AE)

🧠 Concepto

MACsec cifra el tráfico *hop-by-hop* en Layer 2: entre dos switches adyacentes, entre un switch y una laptop, entre dos datacenters unidos por fibra oscura. Analogía: TLS es el sobre sellado que va de tu laptop al servidor de Google. MACsec es el vehículo blindado que transporta ese sobre entre cada par de edificios. TLS protege el contenido; MACsec protege el cable físico en cada tramo.

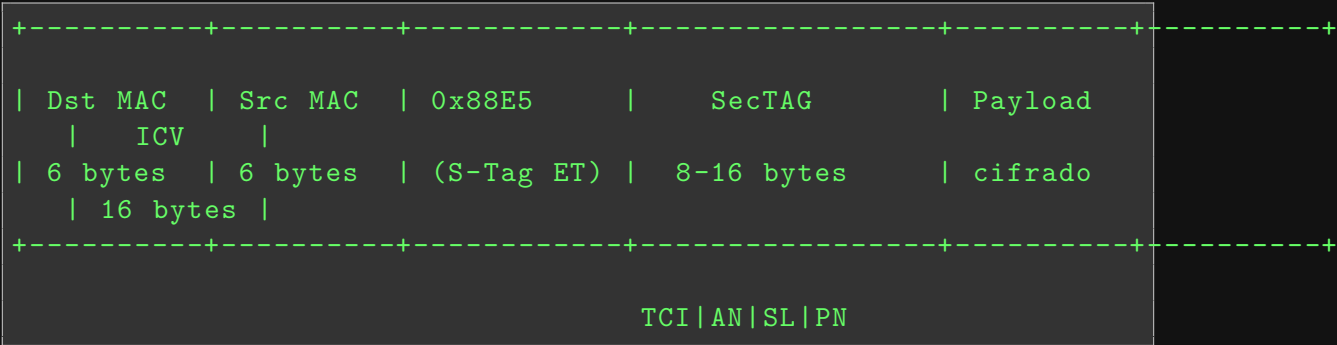
3.2.1. Cómo funciona

1. **MKA (MACsec Key Agreement)**: protocolo basado en 802.1X que negocia claves simétricas entre los peers. Corre sobre el mismo puerto que después

cifrará.

- 2. **SecY (Security Entity):** bloque hardware/software que cifra/descifra cada frame Ethernet en tiempo real.
- 3. **Overhead:** 32 bytes por frame (16 bytes SecTAG + 16 bytes ICV). Latencia adicional: microsegundos en hardware con ASIC dedicado.

3.2.2. Formato del frame MACsec



 **Cuidado**

MACsec requiere soporte en **hardware** (ASIC). No todos los switches lo implementan. Cisco Catalyst 9000 y Nexus 9000 lo soportan nativamente. Switches de gama baja, no.

3.2.3. TLS vs MACsec

Característica	TLS	MACsec
Capa	L7 (aplicación)	L2 (enlace)
Alcance	End-to-end	Hop-by-hop
Qué cifra	Payload TCP/UDP	Frame Ethernet completo
Visible al ISP	Solo IPs	Nada (MACs también ocultas)
Requiere app	Sí	No (transparente a la app)

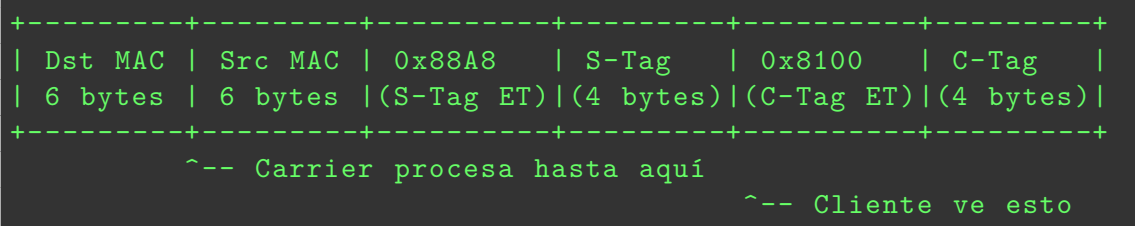
3.3. QinQ — Double Tagging (802.1ad)

 **Concepto**

Problema: un cliente tiene sus propias VLANs (1–4094). El carrier (ISP) también usa VLANs internamente. Si ambos usan VLAN 100, hay colisión: el switch del carrier no distingue “VLAN 100 del Cliente A” de “VLAN 100 del Cliente B”.

QinQ añade un *segundo tag* de VLAN. El tag interno (C-Tag, Customer Tag) pertenece al cliente. El tag externo (S-Tag, Service Tag) pertenece al carrier. El carrier solo ve S-Tags; los C-Tags son opacos para él.

3.3.1. Formato del frame QinQ



Ejemplo real

Telmex te vende metro ethernet para conectar Polanco con Vallejo. Internamente usa S-Tag 500 para identificar tu tráfico. Dentro de tu red usas VLAN 10 (datos) y VLAN 20 (voz). Los switches de Telmex no saben nada de tus VLANs internas: solo mueven frames con S-Tag 500.

3.4. Private VLANs

Concepto

Private VLANs (PVLANS) resuelven el aislamiento *dentro* de una VLAN. Escenario: hosting con 50 VMs del Cliente A en la misma VLAN. No quieres que se vean entre sí — solo deben alcanzar el gateway. Con VLANs normales, cualquier VM habla con cualquier otra. PVLANS lo impiden a nivel de switch, sin necesidad de firewall adicional.

3.4.1. Tipos de puertos

Tipo	Puede hablar con	Caso de uso
Promiscuous	Todos	Router / gateway / firewall
Isolated	Solo Promiscuous	VM aislada, servidor de cliente
Community	Su comunidad + Promiscuous	VMs del mismo tenant que sí se comunican

Configuración / Código

```

vlan 100
  private-vlan primary
vlan 101
  private-vlan isolated
vlan 102
  private-vlan community
vlan 100
  private-vlan association 101,102

! Puerto promiscuous (router/gateway)
interface GigabitEthernet0/1
  
```

```
switchport mode private-vlan promiscuous
switchport private-vlan mapping 100 101,102

! Puerto isolated (VM aislada)
interface GigabitEthernet0/2
switchport mode private-vlan host
switchport private-vlan host-association 100 101
```

3.5. 📡 IGMP Snooping

🧠 Concepto

Sin IGMP Snooping, un switch trata tráfico multicast como **broadcast**: lo manda a todos los puertos del segmento. Un stream de video 4K a 20 Mbps llega a todos, aunque solo una laptop lo haya pedido.

Con IGMP Snooping, el switch *escucha* los mensajes IGMP que cruzan sus puertos y construye una tabla de “qué puerto quiere qué grupo multicast”. El tráfico solo va a los puertos interesados.

3.5.1. Flujo de aprendizaje

1. Router manda **IGMP General Query** a 224.0.0.1 cada 60 s.
2. Host interesado responde con **IGMP Membership Report** (grupo específico).
3. Switch anota: “Puerto X quiere grupo 239.1.1.1”.
4. Host que ya no quiere el grupo manda **IGMP Leave**.
5. Switch elimina la entrada (o espera timeout de membresía).

⚠️ Cuidado

IGMP Snooping necesita un **IGMP Querier** en la VLAN (normalmente el router multicast). Sin Querier, los hosts nunca envían Reports espontáneamente y el switch no aprende → el tráfico multicast termina siendo flooded de todas formas.

Si no hay router multicast, configura `ip igmp snooping querier` en el switch.

3.6. ☁️ Storm Control

🧠 Concepto

Un loop de red o NIC defectuoso puede generar tráfico broadcast/multicast que se multiplica exponencialmente: un frame genera 2, esos 2 generan 4, y en segundos tienes una *broadcast storm* que satura todos los enlaces al 100 %. **Storm Control** monitorea el porcentaje de ancho de banda consumido por broadcast/multicast/unknown unicast. Si supera el umbral, descarta los frames en exceso (o apaga el puerto).

💻 Configuración / Código

```
interface GigabitEthernet0/1
  storm-control broadcast level 20.00
  storm-control multicast level 10.00
  storm-control action shutdown
```

Acción	Comportamiento
drop	Descarta exceso, puerto sigue activo
shutdown	Apaga el puerto (err-disabled)
trap	Genera SNMP trap al NMS

- STP previene loops → evita que la tormenta ocurra.
- Storm Control limita el daño si la tormenta *ya ocurrió*. Son complementarios, no alternativos.

3.7. 🔑 802.1X en Switches — Wired NAC

🧠 Concepto

El mismo 802.1X del Wi-Fi empresarial aplica a **puertos físicos de switch**. Un puerto en estado *unauthorized* no permite tráfico de datos hasta que el dispositivo autentica contra RADIUS. Analogía: el puerto del switch es el torniquete de un edificio. Sin credenciales válidas no entras, aunque físicamente hayas insertado el cable.

3.7.1. MAB — MAC Authentication Bypass

🧠 Concepto

Impresoras, teléfonos IP y cámaras no tienen *supplicant* 802.1X. No pueden iniciar el handshake EAP. **MAB** es el fallback: si el dispositivo no responde al EAPOL-Request inicial, el switch toma su MAC y la envía a RADIUS como si fuera el “usuario”.

RADIUS tiene una base de datos de MACs autorizadas.

Limitación: MACs se pueden falsificar (*MAC spoofing*). MAB no es autenticación fuerte; es “mejor que nada” para dispositivos sin 802.1X.

Configuración / Código

```
aaa new-model
aaa authentication dot1x default group radius
aaa authorization network default group radius
dot1x system-auth-control

interface GigabitEthernet0/2
  switchport mode access
  switchport access vlan 10
  authentication port-control auto
  authentication order dot1x mab
  authentication priority dot1x mab
  dot1x pae authenticator
  mab
```

3.7.2. VLANs de contingencia

- **Guest VLAN:** dispositivos que no inician 802.1X caen aquí (acceso limitado).
- **Auth-Fail VLAN:** intentaron autenticar pero fallaron (portal de error).
- **Critical VLAN:** si RADIUS está caído, el puerto cae aquí en vez de bloquearse por completo. Evita lockout total de la red.

Tecnología Moderna

🌟 **Cisco ISE (Identity Services Engine)** integra 802.1X + MAB + postura de seguridad (¿el dispositivo tiene antivirus? ¿parches aplicados?) + profiling automático (identifica si es impresora HP o iPhone) y asigna políticas por identidad, no por puerto físico. Es el cerebro del NAC empresarial moderno.

Capítulo 4

SDN y OpenFlow

Imagina que cada semáforo de una ciudad toma sus propias decisiones de forma independiente: no sabe qué pasa en el resto de la ciudad, no puede coordinarse con los demás, y para cambiar una política necesitas ir físicamente a cada semáforo. Eso es el networking tradicional. **SDN** pone un controlador central que tiene visibilidad de toda la red y puede programar cada “semáforo” (switch) desde un solo punto.

4.1. El Problema del Networking Tradicional

Concepto

En un router o switch tradicional, el **control plane** (quién decide a dónde va el paquete) y el **data plane** (quién realmente lo reenvía) viven *en el mismo dispositivo*. Esto implica:

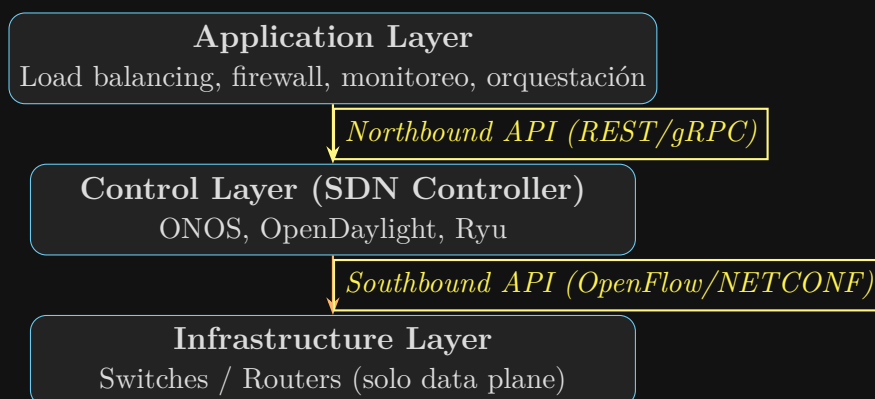
- Para cambiar política de routing, hay que reconfigurar cada dispositivo individualmente (SSH a cada switch, cambiar ACLs, redistribuir rutas).
- No hay visibilidad global: cada router solo conoce su vecindad.
- Nuevas funciones requieren actualizar firmware en *todos* los dispositivos.
- La red no puede adaptarse automáticamente a condiciones cambiantes.

4.2. Arquitectura SDN

Concepto

SDN (Software-Defined Networking) separa el control plane del data plane. El control plane se centraliza en un **SDN Controller**; los switches solo hacen forwarding según las instrucciones del controlador.

4.2.1. Las tres capas



4.2.2. Northbound vs Southbound API

API	Entre	Protocolos típicos
Northbound	Aplicaciones → Controlador	REST, gRPC, GraphQL
Southbound	Controlador → Switches	OpenFlow, NETCONF, gRPC/gNMI
East/West	Controlador → Controlador	OVSDB, clustering protocols

4.3. OpenFlow Protocol

Concepto

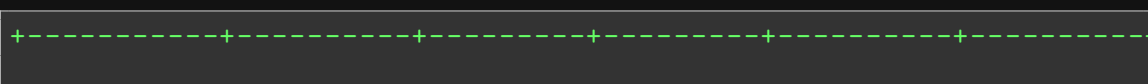
OpenFlow es el protocolo southbound más extendido. El controlador instala *flow entries* en la **flow table** de cada switch OpenFlow. Cuando llega un paquete, el switch lo compara contra las entries y ejecuta las acciones indicadas. Analogía: el switch es un mesero con un menú fijo (flow table). El controlador es el chef que decide qué platos ofrece ese mesero. El mesero no improvisa: si el paquete no está en el menú, lo manda al chef (controller) para que decida.

4.3.1. Flow Entry: Match + Actions

Una flow entry tiene dos partes:

- **Match fields:** los campos del paquete a comparar (puerto de entrada, MAC src/dst, VLAN, IP src/dst, protocolo, puerto TCP/UDP, etc.).
- **Actions:** qué hacer si hay match (forward al puerto X, drop, modificar campo del paquete, encolar, enviar al controlador).

4.3.2. Formato de flow entry OpenFlow 1.3



 Configuración / Código

4.3.3. Pipeline de tablas

OpenFlow 1.1+ permite múltiples flow tables en secuencia:



4.4. Controladores SDN

Controlador	Lenguaje	Caso de uso	Nota
ONOS	Java	Telco, carrier-grade	Clustering nativo
OpenDaylight	Java	Enterprise, modular	Comunidad grande
Ryu	Python	Testing, académico	Fácil de extender
Floodlight	Java	Datacenters	Fork de NOX

4.4.1. OVS — Open vSwitch

🚀 Tecnología Moderna

🚀 **Open vSwitch (OVS)** es la implementación *software* de un switch OpenFlow. Vive en el hypervisor (Linux kernel) y conecta VMs entre sí y con la red física. Es el switch virtual por defecto en OpenStack y muchos entornos cloud privados.

Kubernetes también lo usa: **OVN (Open Virtual Network)** sobre OVS implementa el modelo de red de pods con OpenFlow bajo el capó.

4.5. 🧠 Intent-Based Networking (IBN)

🧠 Concepto

IBN es la evolución de SDN: en vez de programar flows individuales, el admin declara una *intención de alto nivel* y el sistema la traduce automáticamente a reglas de bajo nivel.

Ejemplo de intención:

- “Las VMs de producción nunca deben comunicarse con las de desarrollo.”
- “Todo el tráfico de VoIP debe tener latencia máxima de 20 ms.”
- “Los servidores de la VLAN 100 solo acceden a Internet por el firewall A.”

El controlador verifica continuamente que la intención se cumpla y la restaura automáticamente si algo cambia (*closed-loop automation*).

📄 Ejemplo real

Cisco DNA Center y **Cisco Catalyst Center** son las implementaciones comerciales de IBN de Cisco. **VMware NSX** hace lo mismo para redes virtualizadas de datacenters. En ambos casos, el admin define políticas en una GUI y el sistema genera las ACLs, SGTs (Security Group Tags) y flows de OpenFlow necesarios.

4.6. 📄 SDN en Producción: Realidades

⚠️ Cuidado

El controlador es un **single point of failure**. Si el controlador cae, los switches que dependen de él para tomar decisiones pueden dejar de funcionar (o entrar en modo failopen/failsecure con comportamiento degradado).

Mitigación: clustering de controladores (ONOS tiene clustering nativo), y configurar los switches para seguir forwarding con las flows cacheadas durante

outages breves.

Ventaja SDN	Limitación SDN
Visibilidad global de la red	Controlador = SPOF si no hay HA
Cambios de política centralizados	Latencia extra (paquete → controlador)
Automatización programable	Curva de aprendizaje alta
Desacoplamiento HW/SW	No todos los switches soportan OpenFlow

Capítulo 5



Datacenter: VxLAN y EVPN

Los datacenters modernos tienen un problema que las VLANs clásicas no pueden resolver: millones de tenants (clientes), VMs que migran entre servidores físicos sin cambiar de IP, y redes que deben estirarse entre edificios y regiones geográficas. VxLAN + EVPN es la solución que usan AWS, Google Cloud, Azure y cualquier datacenter serio.

5.1. ⚠ Limitaciones de VLANs en Datacenter

⚠ Cuidado

- **4094 VLANs máximo.** Una nube pública tiene millones de clientes, cada uno necesita su red aislada. 4094 no alcanza ni para empezar.
- **STP no escala.** Con miles de switches, Spanning Tree converge lentamente y bloquea enlaces que podrían estar transmitiendo.
- **VMs migran físicamente.** Cuando vMotion mueve una VM de un servidor a otro, necesita mantener la misma IP (mismo L2). Con VLANs, eso requiere que ambos servidores estén en el mismo dominio L2, lo que limita la distancia física.
- **L2 sobre L3 es complejo.** Los datacenters rutean internamente (L3 escala), pero las aplicaciones esperan L2 (broadcast, ARP). VLANs no resuelven esto.

5.2. 📦 VxLAN — Virtual Extensible LAN

🧠 Concepto

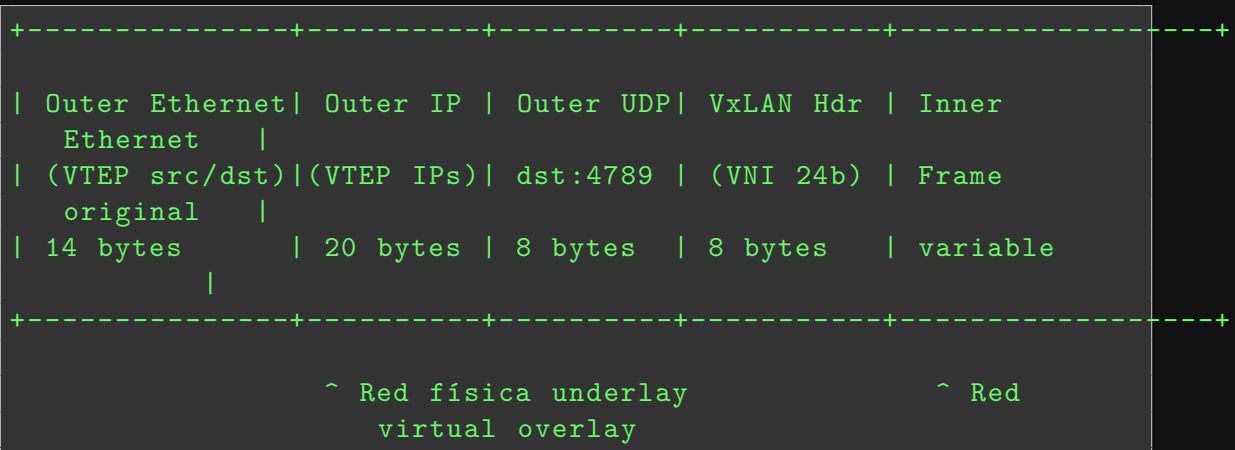
VxLAN crea un *overlay* L2 sobre una red L3. Encapsula frames Ethernet completos dentro de paquetes UDP, permitiendo que dos VMs en servidores físicamente separados (incluso en datacenters distintos) crean que están en el mismo segmento Ethernet.

- **VNI (VxLAN Network Identifier):** 24 bits → 16 millones de seg-

mentos virtuales. Resuelve el límite de 4094 VLANs.

- **VTEP (VxLAN Tunnel Endpoint):** el dispositivo que encapsula/descapsula. Puede ser un switch físico (Leaf) o un servidor Linux con OVS.
- **Puerto UDP:** 4789 (estándar IANA).

5.2.1. Formato del frame VxLAN



- El frame Ethernet original (inner) viaja intacto dentro del túnel UDP.
- La red underlay solo ve IPs de VTEPs; no sabe nada de las VMs.
- **Overhead total:** 50 bytes. En enlaces de 10 GbE, es despreciable.

5.3. Topología Spine-Leaf

Concepto

Las redes de datacenter tradicionales tienen 3 capas: Core → Distribution → Access. Spanning Tree bloquea links redundantes y la latencia varía según el camino.

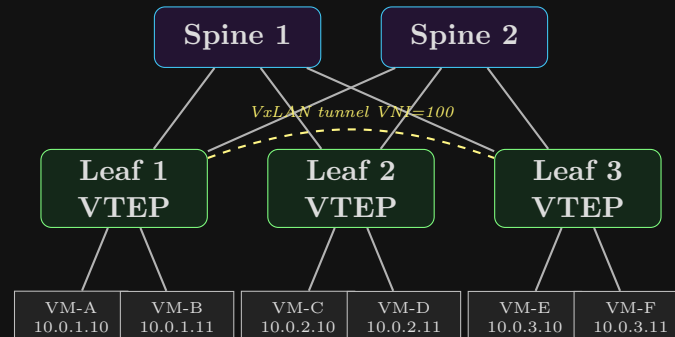
Spine-Leaf reemplaza esas 3 capas con solo 2, eliminando STP y maximizando el uso de todos los enlaces mediante ECMP (Equal-Cost Multi-Path).

5.3.1. Reglas del diseño Spine-Leaf

- Cada **Leaf** conecta a **todos** los Spines (ningún Leaf conecta directamente a otro Leaf).
- Cada **Spine** conecta a **todos** los Leaves (ningún Spine conecta a otro Spine).
- Entre cualquier par de servidores: **máximo 2 hops** (Leaf → Spine → Leaf). Latencia predecible y constante.

- Para escalar: agregar más Leaves (más puertos de servidor) o más Spines (más ancho de banda de east-west).

5.3.2. Diagrama Spine-Leaf con VxLAN



VM-A y VM-E están en el mismo VNI 100 (misma red virtual) aunque en Leaves distintos. El tráfico viaja encapsulado en UDP: Leaf1 → Spine → Leaf3.

5.4. 📡 EVPN — Ethernet VPN

🧠 Concepto

VxLAN sola tiene un problema: ¿cómo sabe un VTEP a qué otro VTEP enviar un frame si no conoce la MAC destino? La respuesta naïve es **flood-and-learn**: inundar el BUM traffic (Broadcast, Unknown-unicast, Multicast) a todos los VTEPs y aprender MACs por el tráfico de respuesta. Eso no escala con miles de VTEPs.

EVPN usa **BGP** como protocolo de control para distribuir información MAC/IP entre VTEPs. Cada VTEP anuncia las MACs/IPs de sus VMs locales vía BGP, y todos los demás VTEPs aprenden esa información *antes* de que llegue el primer paquete. Resultado: no hay flooding innecesario.

5.4.1. Route Types de EVPN

Type	Nombre	Propósito
Type 1	Ethernet A-D Route	Multihoming, failover rápido
Type 2	MAC/IP Advertisement	Aprender MAC + IP de VMs remotas
Type 3	Inclusive Multicast Route	Registro para recibir BUM traffic
Type 4	Ethernet Segment Route	Descubrimiento de multihoming
Type 5	IP Prefix Route	Routing L3 entre VNIs distintos

📄 Ejemplo real

Flujo con EVPN:

1. VM-A (en Leaf1) envía ARP Request buscando VM-E.
2. Leaf1 tiene en su BGP EVPN table: VM-E (MAC=aa:bb:...) está en

VTEP Leaf3 (IP=10.1.1.3), VNI=100. Aprendido vía Type 2 Route.

3. Leaf1 **no floodea**. Responde el ARP directamente (ARP suppression) con la MAC de VM-E desde su tabla local.
4. VM-A envía el frame. Leaf1 lo encapsula en VxLAN hacia Leaf3.
5. Leaf3 desencapsula y entrega a VM-E.

Sin EVPN: el ARP hubiera sido floodado a todos los Leaves. Con EVPN: cero flooding.

5.4.2. Configuración básica (Cisco NX-OS)



Configuración / Código

```
! Habilitar VxLAN y EVPN
feature nv overlay
feature bgp
nv overlay evpn

! NVE interface (VTEP)
interface nve1
  no shutdown
  source-interface loopback0
  member vni 10100
  ingress-replication protocol bgp

! BGP con address-family EVPN
router bgp 65001
  address-family l2vpn evpn
  advertise-pip

neighbor 10.1.1.2 remote-as 65001
  address-family l2vpn evpn
  send-community extended
```

5.5. Routing Asimétrico vs Simétrico en Vx-LAN

Modelo	Dónde rutea	Trade-off
Asimétrico	VTEP de ingreso rutea, VTEP de egreso bridgea	Simple pero cada Leaf necesita to
Simétrico	Ambos VTEPs rutean, L3 VNI dedicado	Más escalable, cada Leaf solo nece



Tecnología Moderna



VxLAN + EVPN en la nube pública: AWS usa una variante propietaria de estos principios internamente. Cada Availability Zone tiene su fabric de Leaf/Spine. El tráfico entre AZs viaja encapsulado (propietario, no OpenFlow). Los Security Groups de AWS se implementan sobre estos mismos mecanismos de overlay.

Capítulo 6



MPLS y QoS

El Internet público trata todos los paquetes igual: si hay congestión, se pierden indiscriminadamente. Eso es inaceptable para voz, video en tiempo real, o los SLAs de un cliente empresarial que paga por un enlace dedicado. **MPLS** permite ingeniería de tráfico precisa; **QoS** garantiza que el tráfico crítico siempre tenga recursos.

6.1. MPLS — Multiprotocol Label Switching

Concepto

En IP normal, cada router examina la dirección destino y hace una búsqueda en su tabla de routing (*longest prefix match*) para cada paquete. MPLS reemplaza esa búsqueda repetida por un sistema de **etiquetas**: el primer router asigna una etiqueta numérica al paquete y los routers intermedios solo hacen *swap* de etiquetas en $O(1)$, sin buscar en tablas de routing completas.

- **LSR (Label Switch Router)**: router en el núcleo MPLS que solo hace swap de etiquetas.
- **LER (Label Edge Router)**: router en el borde que asigna (push) o elimina (pop) etiquetas.
- **LSP (Label Switched Path)**: el camino predeterminado que sigue un flujo de tráfico a través de la red MPLS.
- **FEC (Forwarding Equivalence Class)**: conjunto de paquetes que reciben el mismo tratamiento de forwarding (misma etiqueta, mismo LSP).

6.1.1. Formato de la etiqueta MPLS

31	20 19	16 15	8 7	0
-----+	-----+	-----+	-----+	-----+
Label	Exp	S	TTL	
(20 bits)	(3 b)	(1b)	(8 b)	
-----+	-----+	-----+	-----+	-----+

- **Label (20 bits):** identificador del LSP. Hasta $2^{20} = 1,048,576$ etiquetas.
- **Exp (3 bits):** Traffic Class para QoS (se mapea a DSCP).
- **S (Stack bit):** 1 = última etiqueta del stack (MPLS permite etiquetas anidadas).
- **TTL (8 bits):** igual que el TTL de IP, evita loops.

La etiqueta MPLS se inserta entre el header Ethernet (L2) y el header IP (L3), por eso se llama protocolo de “capa 2.5”.

6.2. 🔄 Operaciones de etiquetas

Operación	Quién la hace	Acción
Push	LER de ingreso	Agrega etiqueta al paquete IP
Swap	LSR en el núcleo	Reemplaza etiqueta por la siguiente del LSP
Pop	LER de egreso (o penúltimo hop)	Elimina etiqueta, entrega IP normal

📄 Ejemplo real

Paquete de 192.168.1.1 → 10.0.0.1 a través de red MPLS de ISP:

1. **LER-A (ingreso):** reconoce FEC “destino 10.0.0.0/8”, hace **push** etiqueta 200. Paquete: [L2][MPLS:200][IP:10.0.0.1][...]
2. **LSR-1:** etiqueta 200 → tabla dice swap a 350 hacia LSR-2. Paquete: [L2][MPLS:350][IP:10.0.0.1][...]
3. **LSR-2:** etiqueta 350 → swap a 120 hacia LER-B.
4. **LER-B (egreso):** hace **pop**, entrega paquete IP puro. Forwarding normal basado en IP hacia 10.0.0.1.

Los LSRs intermedios nunca miraron la IP destino. Solo swapearon etiquetas.

6.3. 🛠️ MPLS Traffic Engineering (TE)

🧠 Concepto

Con routing IP normal, el tráfico siempre toma el camino de menor costo (shortest path). Si hay un enlace congestionado pero otro libre con mayor costo, IP lo ignora. MPLS-TE permite **forzar** LSPs por caminos específicos, balancear carga por múltiples rutas, y garantizar ancho de banda reservado para flujos críticos.

El protocolo de señalización para MPLS-TE es **RSVP-TE** (Resource Reservation Protocol - Traffic Engineering), que reserva recursos a lo largo del LSP.

 Tecnología Moderna

 **Segment Routing (SR-MPLS):** la evolución moderna de MPLS-TE. En lugar de señalización RSVP hop-a-hop, el router de ingreso codifica el camino completo como un *stack de etiquetas* (segment list). No hay estado de LSP en los routers intermedios. Google, Facebook y los grandes ISPs migraron de RSVP-TE a Segment Routing por su simplicidad operacional.

6.4. ⚖️ QoS — Quality of Service

 Concepto

QoS es el conjunto de mecanismos que garantiza comportamiento predecible para tráfico crítico en una red compartida. Sin QoS, todos los paquetes compiten igual por ancho de banda y, en congestión, cualquier flujo puede perder paquetes.

6.4.1. Los tres problemas que QoS resuelve

 Cuidado

- **Bandwidth:** un flujo puede consumir todo el ancho de banda disponible (Netflix descargando vs. llamada VoIP en el mismo enlace).
- **Delay (latencia):** colas llenas aumentan la latencia. Voz tolera máximo 150ms one-way; un backup FTP puede tolerar segundos.
- **Jitter (variación de latencia):** paquetes de voz deben llegar con intervalos regulares. Jitter alto causa distorsión de audio.

6.4.2. Modelo DiffServ y DSCP

 Concepto

DiffServ (Differentiated Services) es el modelo dominante de QoS en Internet y redes empresariales. Cada paquete IP lleva un campo **DSCP (Differentiated Services Code Point)** de 6 bits en el header IP (parte del campo ToS/Traffic Class). Los routers aplican tratamiento diferenciado según ese valor, sin mantener estado por flujo.

Clase	DSCP	Valor	Uso típico
EF (Expedited Forwarding)	EF	46 (101110)	VoIP, videoconferencia
AF41 (Assured Forwarding)	AF41	34 (100010)	Video interactivo
AF31	AF31	26 (011010)	Video streaming
AF21	AF21	18 (010010)	Transacciones críticas
AF11	AF11	10 (001010)	Datos de negocios
CS0 (Best Effort)	BE	0 (000000)	Tráfico normal

6.4.3. Mecanismos de QoS



- **LLQ (Low Latency Queue):** cola de alta prioridad para VoIP/video. Se sirve primero, siempre. Garantiza latencia mínima.
- **CBWFQ (Class-Based WFQ):** asigna porcentajes de ancho de banda garantizado por clase. Si nadie más usa su porción, puede tomar el exceso.
- **WRED (Weighted Random Early Detection):** descarta paquetes de baja prioridad antes de que la cola se llene, evitando TCP global synchronization.
- **Token Bucket:** modelo para policing/shaping. Se “acumulan fichas” a una tasa CIR (Committed Information Rate); cada paquete consume fichas. Sin fichas → paquete se descarta (policing) o se retrasa (shaping).

Ejemplo real

QoS en una empresa con 100 Mbps hacia Internet:

- VoIP (DSCP EF): LLQ, 10 Mbps garantizados, latencia <10ms.
- Videoconferencia (DSCP AF41): CBWFQ, 30 Mbps garantizados.
- CRM/ERP (DSCP AF21): CBWFQ, 25 Mbps garantizados.
- Navegación general (DSCP BE): 35 Mbps restantes, WRED activo.

Si las llamadas VoIP no usan sus 10 Mbps, el resto puede aprovecharlos. Pero si VoIP necesita esos 10 Mbps, los tiene garantizados sin importar lo demás.

Tecnología Moderna

 **QoS en la nube:** AWS, GCP y Azure ofrecen Enhanced Networking (SR-IOV) para reducir latencia en instancias. Google Cloud usa **Andromeda**, su stack de virtualización de red que aplica QoS a nivel de hipervisor. En

Kubernetes, los `LimitRange` y `ResourceQuota` son la capa de QoS de aplicación; el QoS de red real requiere CNI plugins como Cilium con bandwidth enforcement via eBPF.

Capítulo 7

Networking en Kubernetes

Kubernetes es hoy la plataforma estándar de orquestación de contenedores. Su modelo de red es intencionalmente simple en la interfaz pero complejo en la implementación: *todo Pod puede hablar con todo Pod sin NAT*. Entender cómo se implementa eso es fundamental para operar clusters en producción.

7.1. El Modelo de Red de Kubernetes

Concepto

Kubernetes impone cuatro reglas fundamentales al networking:

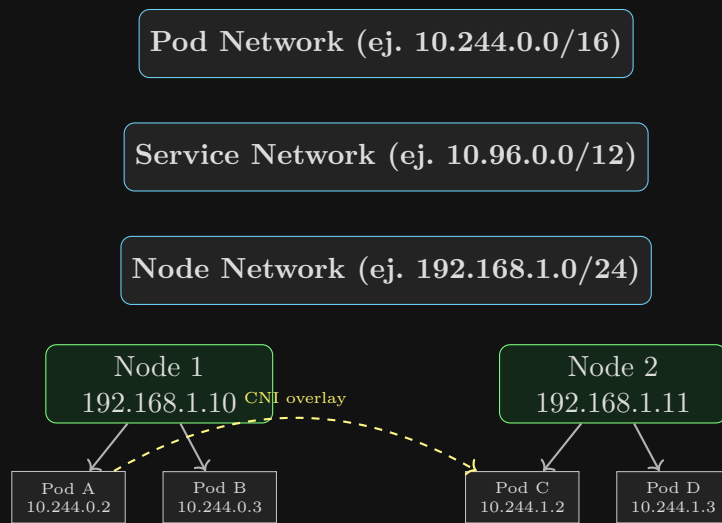
1. Cada Pod tiene su propia dirección IP única en el cluster.
2. Todos los Pods pueden comunicarse entre sí sin NAT.
3. Los nodos pueden comunicarse con todos los Pods sin NAT.
4. La IP que ve un Pod de sí mismo es la misma que ven los demás Pods.

Estas reglas simplifican el modelo: las aplicaciones no necesitan saber si están en el mismo nodo o en nodos distintos. El networking subyacente lo resuelve.

Cuidado

Kubernetes **define** el modelo pero **no lo implementa**. La implementación la provee el **CNI plugin** (Container Network Interface): Cilium, Calico, Flannel, Weave, etc. Distintos CNIs tienen distintas capacidades (Network Policies, encryption, eBPF, multicast, etc.).

7.2. Tipos de red en un cluster



- **Pod Network:** rango de IPs asignadas a Pods. CNI lo administra.
- **Service Network:** IPs virtuales de Services (ClusterIP). Solo existen en iptables/eBPF; ningún dispositivo real las tiene.
- **Node Network:** red física/real de los nodos. Preexiste a Kubernetes.

7.3. Services y kube-proxy

Concepto

Los Pods son efímeros: mueren y resurgen con nuevas IPs. Un **Service** es una abstracción estable: tiene una IP virtual fija (ClusterIP) y un DNS name, y hace load balancing hacia los Pods seleccionados por su **selector**. **kube-proxy** implementa los Services programando reglas en el kernel del nodo:

Modo	Mecanismo	Rendimiento
userspace	Proxy en espacio de usuario	Lento, legado
iptables	Reglas NAT en iptables	Bueno, O(n) rules
IPVS	IP Virtual Server del kernel	Mejor, O(1) lookup
eBPF	Cilium sin kube-proxy	Óptimo, bypass kernel stack

7.3.1. Tipos de Service

- **ClusterIP (default):** IP virtual solo accesible dentro del cluster. Uso: comunicación interna entre microservicios.
- **NodePort:** expone el Service en un puerto fijo de cada nodo (30000–32767). Accesible desde fuera del cluster via **NodeIP:NodePort**.

- **LoadBalancer:** solicita un load balancer externo al cloud provider (AWS ELB, GCP LB). Asigna IP pública automáticamente.
- **ExternalName:** alias DNS a un FQDN externo. Útil para integrar servicios externos como si fueran internos.
- **Headless (ClusterIP: None):** no hay IP virtual; DNS devuelve directamente las IPs de los Pods. Usado por StatefulSets (Kafka, Cassandra).

7.4. 🛡️ Network Policies

🧠 Concepto

Por defecto, todos los Pods en un cluster pueden comunicarse entre sí (modelo flat). **NetworkPolicy** permite definir reglas de firewall a nivel de Pod: qué pods pueden conectarse a qué otros pods, en qué puertos y protocolos. Las NetworkPolicies son aditivas: si no hay política, se permite todo. Si hay al menos una política que selecciona un Pod, ese Pod solo acepta lo que las políticas permiten.

💻 Configuración / Código

```
# Solo permitir trafico al pod "backend" desde pods con
  label "tier: frontend"
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-ingress
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          tier: frontend
    ports:
    - protocol: TCP
      port: 8080
```

⚠️ Cuidado

NetworkPolicies requieren que el CNI plugin las soporte. Flannel no las implementa. Calico, Cilium, y Weave sí. Sin un CNI compatible, los objetos NetworkPolicy se crean pero no tienen efecto alguno — la red sigue siendo

completamente abierta.

7.5. Ingress y Gateway API

Concepto

Los Services de tipo LoadBalancer son costosos (un LB externo por Service). **Ingress** consolida el tráfico HTTP/HTTPS de múltiples Services detrás de un solo load balancer, enrutando por host y path.

Configuración / Código

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: mi-app-ingress
spec:
  rules:
    - host: api.miempresa.com
      http:
        paths:
          - path: /v1
            pathType: Prefix
            backend:
              service:
                name: api-v1
                port:
                  number: 80
          - path: /v2
            pathType: Prefix
            backend:
              service:
                name: api-v2
                port:
                  number: 80
```

Tecnología Moderna

 **Gateway API** (sucesor de Ingress): API más expresiva con soporte nativo para TCP, UDP, gRPC, y modelos multi-tenant. Separa responsabilidades entre el equipo de infraestructura (GatewayClass, Gateway) y el equipo de aplicación (HTTPRoute, TCPRoute). Cilium, Istio y Nginx ya la implementan. Ingress quedará deprecated eventualmente.

7.6. 📡 CNI Plugins: Cilium vs Calico

Feature	Cilium	Calico
Data plane	eBPF	iptables / eBPF (opcional)
Rendimiento	Excelente	Bueno
Network Policies	L3/L4/L7 (HTTP, gRPC)	L3/L4
Observabilidad	Hubble (flujos)	Limitada
Modo sin kube-proxy	Sí (eBPF)	Parcial
BGP support	Sí	Sí (más maduro)
Encryp. transparente	WireGuard/IPsec	WireGuard/IPsec

📖 Ejemplo real

Por qué Cilium usa eBPF: Con iptables, cada Service agrega reglas que el kernel evalúa secuencialmente. Con 10,000 Services, kube-proxy crea cientos de miles de reglas; la actualización tarda minutos y la evaluación es $O(n)$. eBPF usa mapas hash: lookup $O(1)$, actualizaciones atómicas en microsegundos. En clusters grandes (>500 nodos), Cilium reduce la latencia de Service lookup de milisegundos a microsegundos.

Capítulo 8

Cloud Networking

Las nubes públicas no son simplemente servidores remotos con Internet rápido. Son redes enteras diseñadas desde cero para escala masiva, multi-tenancy y automatización via API. Entender su arquitectura diferencia a un ingeniero que *usa* la nube de uno que *diseña* sobre ella.

8.1. VPC — Virtual Private Cloud

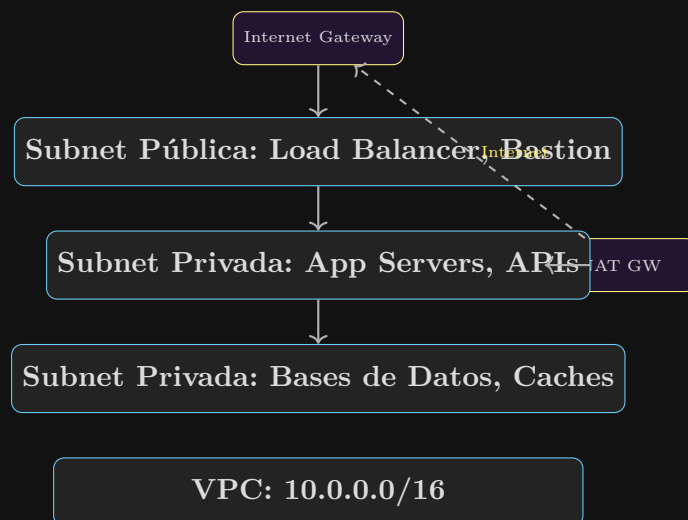


Concepto

Una **VPC (Virtual Private Cloud)** es una red virtual aislada dentro de la nube pública. Cada cliente tiene su propia VPC con su propio espacio de direcciones IP, tablas de routing, y políticas de seguridad. El aislamiento entre VPCs lo garantiza el hypervisor/SDN del proveedor, no hardware dedicado. Componentes fundamentales (nomenclatura AWS, análogos en GCP/Azure):

- **CIDR block:** rango IP de la VPC (ej. 10.0.0.0/16).
- **Subnets:** subdivisiones de la VPC, asociadas a una zona de disponibilidad.
- **Route Tables:** controlan a dónde va el tráfico de cada subnet.
- **Internet Gateway (IGW):** permite que subnets públicas accedan a Internet.
- **NAT Gateway:** permite que subnets privadas salgan a Internet sin ser directamente accesibles.
- **Security Groups:** firewall stateful por instancia (como iptables).
- **NACLs (Network ACLs):** firewall stateless por subnet.

8.1.1. Arquitectura típica multi-tier



- Subnet pública: accesible desde Internet (IGW en route table).
- Subnet privada de app: no accesible directamente, sale a Internet vía NAT.
- Subnet privada de DB: sin acceso a Internet en ninguna dirección.

8.2. ☞ Conectividad entre VPCs y On-Premise

8.2.1. VPC Peering

🧠 Concepto

Conexión privada entre dos VPCs (mismo o distinto account, misma o distinta región). El tráfico viaja por la backbone del proveedor, no por Internet. **No transitivo**: si A está peered con B, y B con C, A **no** puede hablar con C a través de B.

8.2.2. Transit Gateway / Cloud Router

🧠 Concepto

Solución hub-and-spoke para conectar muchas VPCs y redes on-premise. En lugar de N^2 peerings, cada VPC conecta al Transit Gateway (AWS) o Cloud Router (GCP). Sí es transitivo. Permite routing más granular y segmentación de tráfico.

8.2.3. VPN y Direct Connect / Dedicated Interconnect

Opción	Características	Cuándo usar
VPN (IPsec)	Sobre Internet, encriptado, rápido de desplegar	Dev/test, fallback
Direct Connect	Enlace físico dedicado, bajo jitter, SLA fuerte	Producción crítica
PrivateLink	Acceso a servicios del proveedor sin Internet	S3, RDS, APIs internas

8.3. 📊 Load Balancing en la nube

🧠 Concepto

Los proveedores cloud ofrecen load balancers gestionados que escalan automáticamente. La elección depende del tipo de tráfico:

Tipo	Capa OSI	AWS	GCP
L4 TCP/UDP	Transporte	Network LB (NLB)	TCP/UDP LB
L7 HTTP/HTTPS	Aplicación	Application LB (ALB)	HTTP(S) LB
Global anycast	L7 global	CloudFront + ALB	Global HTTP LB

📁 Ejemplo real

ALB (Application Load Balancer) puede enrutar por:

- Host header: `api.ejemplo.com` → Target Group A;
`admin.ejemplo.com` → Target Group B.
- Path: `/api/v1` → servicio legacy; `/api/v2` → servicio nuevo.
- Header HTTP: `X-Version: beta` → 10% canary deployment.
- Peso: 90% tráfico a stable, 10% a canary (weighted target groups).

8.4. 🛡️ Seguridad de Red en la Nube

8.4.1. Security Groups vs NACLs

Característica	Security Groups	NACLs
Estado	Stateful (respuesta automática)	Stateless (reglas explícitas ida y vuelta)
Nivel	Instancia (ENI)	Subnet
Reglas	Solo allow	Allow y deny
Evaluación	Todas las reglas juntas	Numeradas, primera que aplica
Uso típico	Firewall de host	Bloqueo de IPs maliciosas

⚠ Cuidado

NACLs son stateless: si permites tráfico entrante al puerto 80, necesitas una regla de salida para los puertos efímeros (1024–65535) para que las respuestas puedan salir. Con Security Groups no: al ser stateful, la respuesta se permite automáticamente.

8.4.2. AWS PrivateLink / GCP Private Service Connect**🚀 Tecnología Moderna**

🚀 Acceso a servicios (S3, DynamoDB, APIs de terceros) sin que el tráfico salga a Internet. El servicio se expone como un endpoint en tu VPC con IP privada. El tráfico nunca cruza un IGW. Mitiga riesgo de exfiltración y elimina cargos de NAT Gateway para tráfico a servicios del proveedor (S3 puede ser millones al mes).

8.5. 🛠 CDN y Edge Networking**🧠 Concepto**

Una **CDN (Content Delivery Network)** distribuye contenido estático (imágenes, JS, CSS, videos) a **PoPs (Points of Presence)** cercanos a los usuarios finales. El objetivo: reducir latencia sirviendo desde el PoP más próximo al usuario, no desde el origen.

Provider	CDN	Diferenciador
AWS	CloudFront	Integración nativa con S3, ALB, Lambda@Edge
GCP	Cloud CDN	Backbone privado Google, anycast global
Cloudflare	Cloudflare CDN	DDoS protection, Workers edge compute
Akamai	Akamai CDN	Mayor red de PoPs del mundo

📖 Ejemplo real

Latencia con y sin CDN para usuario en CDMX accediendo a servidor en Virginia:

- Sin CDN: CDMX → Internet → Virginia. RTT ≈ 80ms. Carga HTML+JS+CSS requiere múltiples round-trips: >500ms.
- Con CDN (PoP en CDMX): JS/CSS servidos localmente. RTT al PoP ≈ 5ms. Solo llamadas a API van a Virginia. Carga percibida: <100ms.

Capítulo 9

Multicast

Imagina transmitir un partido de fútbol a 1 millón de espectadores. Con **unicast**, el servidor enviaría 1 millón de copias del mismo stream: imposible. Con **broadcast**, llegaría a todos los dispositivos de la red, incluso a quienes no quieren verlo. **Multicast** resuelve esto elegantemente: el servidor envía *una sola copia* que la red replica solo hacia quienes la solicitaron.

9.1. Unicast vs Broadcast vs Multicast

Modelo	Destinatario	Copias en red	Uso
Unicast	Un host específico	Una por destino	HTTP, SSH, DNS
Broadcast	Todos en el segmento L2	Una (pero llega a todos)	ARP, DHCP
Multicast	Grupo de interesados	Una (la red la replica)	IPTV, routing protocols
Anycast	El más cercano del grupo	Una	DNS raíz, CDN

Cuidado

Multicast tiene reputación de ser complejo de implementar y operar. No es mito: requiere soporte en cada router del camino, gestión de grupos, y protocolos adicionales (PIM, IGMP, MSDP). Por eso en la nube pública casi no existe multicast nativo: AWS y GCP no lo soportan en VPCs. Se reemplaza con overlay multicast (software) o CDN.

9.2. Direcciones Multicast

Concepto

IPv4 reserva el rango **224.0.0.0/4** para multicast (224.x.x.x a 239.x.x.x). Cada *grupo multicast* tiene una dirección en este rango. Un host se “une” al grupo para recibir tráfico; puede abandonarlo en cualquier momento.

Rango	Nombre	Uso
224.0.0.0/24	Link-local	Protocolos de routing (OSPF: 224.0.0.5/6, HSRP: 224.0.0.2)
224.0.1.0/24	Internetwork	NTP: 224.0.1.1, uso general
232.0.0.0/8	Source-Specific	SSM (Source-Specific Multicast)
239.0.0.0/8	Admin-scoped	Multicast privado dentro de organización

9.2.1. MAC Multicast

🧠 Concepto

Cuando un router L2 recibe un paquete multicast IP, lo encapsula en un frame Ethernet con una MAC de destino especial. El bit I/G del primer byte de la MAC indica multicast. Para IPs multicast, la MAC se construye así:

01:00:5E:[0] [últimos 23 bits de la IP multicast]

Ej: IP 224.1.1.1 → MAC 01:00:5E:01:01:01.

Consecuencia: los switches L2 no hacen multicast por defecto — inundan a todos los puertos como si fuera broadcast. **IGMP Snooping** soluciona esto.

9.3. 🌽 IGMP — Gestión de Grupos

🧠 Concepto

IGMP (Internet Group Management Protocol) es el protocolo entre hosts y el router de primer salto para gestionar membresía a grupos multicast.

Versión	Característica clave	Uso
IGMPv1	Join básico, no hay Leave	Obsoleto
IGMPv2	Leave Group message	Aún común
IGMPv3	Source filtering (SSM)	Moderno, requerido para SSM

- **IGMP Query:** el router pregunta “¿alguien quiere grupos multicast?” cada 60–125 segundos.
- **IGMP Report:** el host responde con los grupos a los que pertenece.
- **IGMP Leave:** el host notifica que abandona un grupo (IGMPv2+).
- **IGMP Snooping:** los switches L2 “escuchan” mensajes IGMP y aprenden qué puerto tiene interesados en cada grupo, evitando flooding innecesario.

9.4. 🌲 PIM — Routing Multicast entre Routers

🧠 Concepto

IGMP maneja el último salto (host $\leftrightarrow$ router). Para que el tráfico multicast fluya entre routers en la red, se usa **PIM (Protocol Independent Multicast)**. “Protocol Independent” porque funciona sobre cualquier protocolo de routing unicast (OSPF, BGP, etc.), usando su tabla de routing para construir árboles de distribución multicast.

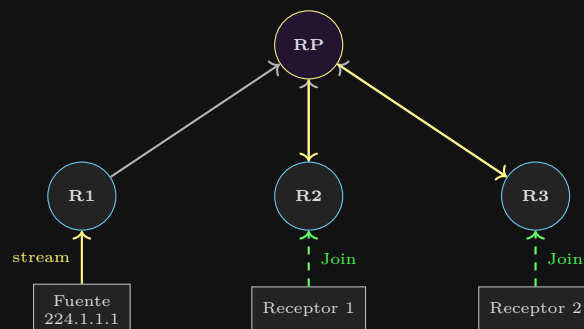
9.4.1. PIM Dense Mode (PIM-DM)

- Asume que todos los segmentos quieren el tráfico multicast.
- **Flood and prune:** inunda todo, luego los routers sin interesados envían mensajes Prune para cortar el árbol.
- Simple pero ineficiente. Solo adecuado para redes pequeñas donde la mayoría de los segmentos sí tienen receptores.

9.4.2. PIM Sparse Mode (PIM-SM)

🧠 Concepto

Modelo opuesto: asume que *pocos* segmentos quieren el tráfico. Los routers deben suscribirse explícitamente. Usa un **RP (Rendezvous Point)**: router central donde fuentes y receptores se “encuentran”.



Receptores hacen Join hacia el RP. Fuente registra su stream en el RP. RP distribuye a los routers con receptores. Después puede formarse un Shortest Path Tree directo fuente $\rightarrow$ receptor (SPT switchover).

9.4.3. PIM Source-Specific Multicast (PIM-SSM)

🧠 Concepto

Variante moderna que elimina el RP. Los receptores se suscriben a un grupo multicast *y* una fuente específica: (S, G) en lugar de (*, G). Más simple, más seguro (no hay fuentes no autorizadas), y usa el rango 232.0.0.0/8. Requiere IGMPv3 en los hosts.

9.5. 📺 Aplicaciones Reales

📖 Ejemplo real

IPTV (Televisión sobre IP): operadores como Izzi, Totalplay o Claro TV distribuyen canales de TV usando multicast. Un canal de HD requiere $\approx 4\text{--}8$ Mbps. Con unicast, 10,000 suscriptores viendo el mismo canal = 80 Gbps desde el servidor. Con multicast = 8 Mbps independientemente del número de suscriptores. El router del cliente (CPE) hace IGMP Join cuando el usuario cambia de canal.

📖 Ejemplo real

Protocolos de routing: OSPF usa 224.0.0.5 (todos los routers OSPF) y 224.0.0.6 (DR/BDR). Esto es multicast link-local: el paquete nunca pasa de ese segmento L2. Sin multicast, OSPF tendría que usar broadcast o unicast punto a punto.

🚀 Tecnología Moderna

🚀 **Multicast en datacenters modernos:** VxLAN con BUM traffic (Broadcast, Unknown-unicast, Multicast) puede usar multicast IP underlay en lugar de ingress replication. El switch Leaf hace IGMP Join para el grupo multicast del VNI. Sin embargo, EVPN con ingress replication es hoy más común porque no requiere multicast en el underlay (muchos clouds no lo soportan). En Kubernetes, **no hay multicast nativo en las VPCs de nube pública:** se simula con servicios como AWS SNS (fan-out de mensajes) o con aplicación-level multicast en frameworks como Hazelcast.

9.6. ✅ Resumen del Ecosistema Parte 2

Al completar estos apuntes has cubierto el stack completo del networking moderno:

Tema	Dónde aplica hoy
Wireless empresarial	Oficinas, campus, IoT industrial
Switching avanzado	Cualquier red LAN empresarial
SDN / OpenFlow	Datacenter, WAN empresarial
VxLAN + EVPN	AWS, Azure, GCP internamente; datacenter privado
MPLS + QoS	ISPs, WAN empresarial, voz sobre IP
Kubernetes networking	Todo cluster en producción
Cloud networking	Toda aplicación en AWS/GCP/Azure
Multicast	IPTV, protocolos de routing, streaming masivo